

Prereq: CI/CD Foundations

Up next in Hands-on Labs →

**Lab 02: Build & Test**

 Hands-on lab · 45 min · Env: GitHub + Actions · Overview & prereqs · Download PDF ·

Reset checks

CI/CD ENGINEERING · KNOWLEDGE BASE

# Lab 01: First Pipeline

CICD BY Nabawy

Reader editionNabawy

# Objective

Ship a green CI pipeline on GitHub Actions: push → build → test → status check on the PR.

## Prerequisites

- A GitHub repo you own; Node 20+ locally (any tiny JS project works, even `package.json` + one test file).
- Concepts: CI/CD Pipelines, GitHub Actions.

## Architecture

Push → GitHub webhook → hosted runner (ubuntu-latest) → checkout → setup-node → `npm ci` → `npm test` → green check on PR.

## Setup

Repo with `package.json` containing a test script (even `"test": "node --test"`).

## Step 1 — Add the workflow

### Execute

Create `.github/workflows/ci.yml`:

```
name: ci on: push: pull_request: jobs: build: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v4 with: { node-version: 20, cache: npm } - run: npm ci - run: npm test
```

Push to a branch, open a PR.

## Expected Result

Actions tab shows the run green; the PR shows a green check.

## Why

This is the smallest closed loop: trigger → runner → job → feedback on the PR (see pipeline anatomy).

## Step 2 — Require the check

### Execute

Repo Settings → Branches → protect main → require build status check. Push a breaking change to a PR.

## Expected Result

Merge button blocked until green.

## Why

An unenforced gate is decoration; branch protection makes CI a rule (see PR gates).

## Verification

- [ ] PR shows green check from the workflow.
- [ ] Merging to main without green is blocked.
- [ ] Breaking the test turns the PR red.

## Failure Scenarios

- `npm ci` fails: lockfile missing or out of sync — run `npm install` locally, commit the lockfile.
- Runner queues long: free-tier concurrency; retry, or shrink the job.

## Cleanup

Keep the workflow — every later lab extends it.

## What This Proved

Triggers fire pipelines, runners execute jobs, checks gate merges. Continue with Lab 02.

## Interview Notes

- What the green check actually proves — and which misconfiguration lets red code merge anyway.
- Why breaking the test on purpose is the real verification, not the green run.

RELATED BOOKS



Lab 02: Build & Test

45 min · INT

→ Next in path



Lab 01: GitHub Webhook Trigger for Jenkins

45 min · INT

→ Same path



Lab 02: Docker Build & Push with Jenkins

50 min · INT

→ Same path



Lab 03: Artifacts

45 min · INT

→ Same path